

FitPal: Repetition-Level Muscle-Activation Feedback Surface EMG on an M5Stack Core2 with Per-User Calibration

Juan Fernando Castaño Diab Singer Román Villarreal

Correspondence: jfc9577@nyu.edu

Abstract

A repetition counter driven by one EMG threshold works for one person on one day. Electrode placement, skin impedance and muscle mass change the amplitude several-fold between users, and mains hum or a knocked cable is enough to produce a count. FitPal is a wearable surface-EMG device that tells a lifter, repetition by repetition, whether the target muscle was actually worked. A Grove EMG sensor feeds an M5Stack Core2, which shows an “effective” or “not effective” screen, vibrates, and streams results to an Arduino IoT Cloud dashboard. The firmware samples at 1 kHz in a dedicated FreeRTOS task, applies a 50 Hz notch and 20–350 Hz band limits, computes a 100 ms RMS envelope, expresses it as a fraction of the user’s maximum voluntary contraction (%MVC) measured in a six-second calibration, and detects repetitions with a hysteresis detector and a duration gate. The whole signal chain is one hardware-independent header that is compiled and tested on a PC; its envelope agrees with an independent Python implementation to 0.0076 mV and both return the same ten repetitions with the same flags. On simulated sessions containing mains hum, DC offset, baseline wander, motion spikes and a $\pm 35\%$ spread in subject gain, the chain reaches $F1 = 0.995$ over 60 sessions, against 0.563 for a fixed-threshold detector. The signal model is our own, so these scores describe the algorithm and not people. The sketch compiles for the Core2 with the ESP32 Arduino core. Scoring on recordings from the physical sensor, and a run on the device, are still to be done.

1 Introduction

A lifter usually cannot tell whether a repetition loaded the intended muscle. Machines and coaches give indirect cues; surface electromyography (EMG) measures the electrical activity of the muscle itself. The difficulty is that a raw EMG voltage is not a reading. It is a noise-like signal, between tens of microvolts and a few millivolts at the skin, whose *power* follows muscle activation, and it sits on top of a DC offset, baseline wander, mains interference and motion artefacts that are often larger than the signal.

We ask two questions about a small embedded system that turns that voltage into a verdict.

Question 1: can the verdict be made independent of the user? Amplitude differs between people, between electrode placements and between days. A threshold in millivolts is therefore a property of one session. The system has to measure each user’s own range and judge every repetition against it.

Question 2: how do we know the firmware does what we say it does? A device on a microcontroller is hard to inspect, and a subject is not always available. We want a procedure in which the code that runs on the board is the code that is tested, and in which the test signal contains the artefacts that break simple detectors.

What we find. Per-user normalisation by a six-second calibration removes the dependence on gain: over 60 simulated sessions with a $\pm 35\%$ gain spread, the chain finds every effective repetition and miscounts five, while a fixed threshold finds fewer than half of them. The signal chain is a single header with no Arduino dependency, so the same file is built into the firmware and into the PC test; the two agree to 0.0076 mV. We have not scored real recordings, and the simulation is our own model, so we claim the algorithm is correct and robust to the listed artefacts, nothing more.

Contributions. We do not propose a new EMG method; every stage is standard. The contributions are the following.

1. A complete embedded chain, from ADC to an effective/not-effective flag, with the reasoning behind each parameter (Section 3).
2. A calibration procedure of about six seconds that expresses activation as a fraction of the user's own maximum and fails visibly when the electrode is off or the user does not contract.
3. A firmware structure in which a 1 kHz sampler task is decoupled from the display and the cloud connection.
4. A verification pipeline in which the device header is compiled on a PC and compared with an independent implementation, plus a simulated evaluation with ground truth (Section 4).
5. Code, simulation and raw figures in one repository (Appendix B).

Organisation. Section 2 gives background on surface EMG. Section 3 describes the hardware and the signal chain, Section 4 the verification protocol, Section 5 the results, and Section 6 what they do and do not establish.

2 Background

Surface EMG. The signal recorded over a muscle is the sum of motor-unit action potentials. Its useful energy lies roughly between 20 and 450 Hz (De Luca, 1997; Merletti and Parker, 2004), and its amplitude, not its instantaneous value, tracks effort. Estimating that amplitude from a short window, for example a moving RMS, is standard practice, and the choice of window and of noise suppression affects the estimate (Clancy et al., 2002).

Normalisation. Raw amplitude cannot be compared across people or sessions, so it is usually expressed relative to a reference contraction, most often a maximum voluntary contraction (MVC). The method has known weaknesses, since an MVC depends on motivation and technique, but it remains the common choice for healthy participants (Burden, 2010). We use it because it needs no equipment beyond the device itself.

Interference and electrode practice. Mains pickup appears as a narrow line at 50 or 60 Hz; baseline wander and motion artefact sit below about 20 Hz. Sensor and electrode placement recommendations are collected in the SENIAM guidelines (Hermens et al., 2000). The Core2's ESP32 converter is noisy and nonlinear, which is why we average four reads per sample and use the calibrated millivolt conversion.

Fatigue. A falling peak amplitude across a set is one simple indicator of fatigue; spectral indicators are more specific but need longer windows and more computation (Cifrek et al., 2009). We use only the first.

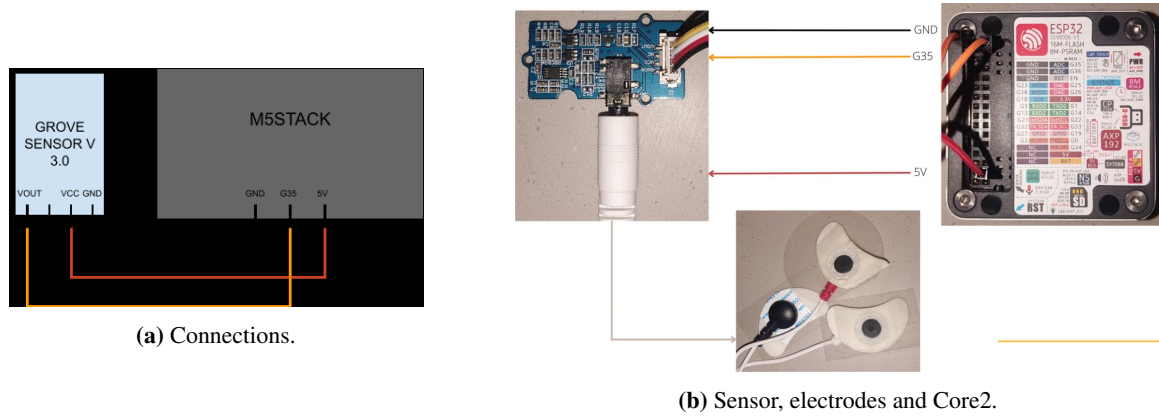


Figure 1: Hardware.

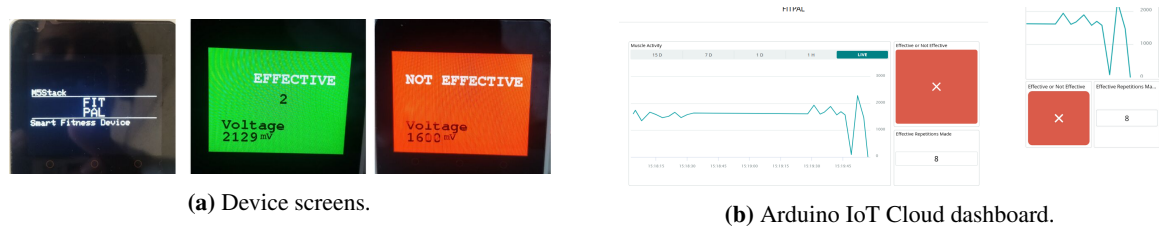


Figure 2: User interface on the device and in the cloud.

3 System and Signal Chain

3.1 Hardware

The sensor is a Grove EMG module. Its VOUT pin goes to pin G35 of the M5Stack Core2, VCC to 5 V and GND to GND; the electrodes connect to the module through a TRS plug (Figure 1). The Core2 supplies the display, three buttons, a vibration motor and Wi-Fi.

3.2 Signal chain

Samples are processed once per millisecond in this order (Figure 4).

Acquisition. A dedicated FreeRTOS task on core 0 wakes every millisecond with `vTaskDelayUntil`. Each sample is the mean of four `analogReadMilliVolts` reads, so every later value is in real millivolts.

Mains notch. A biquad notch with $Q = 30$ at the grid frequency (50 Hz by default; a constant selects 60 Hz) with coefficients from the audio-EQ cookbook (Bristow-Johnson). It removes hum without cutting into the EMG band.

High-pass, 20 Hz, and low-pass, 350 Hz. Second-order Butterworth sections ($Q = 1/\sqrt{2}$). The high-pass removes the sensor's DC offset, baseline wander and slow motion artefact; the low-pass stays well clear of the 500 Hz Nyquist limit. Because the high-pass sees the DC offset as a step, the first 500 ms of data are discarded so the filter settles before any envelope is computed.



Figure 3: The device during a biceps curl: electrodes on the arm, Core2 held in the other hand and resting on the bench. Frames from demonstration footage; this footage is not part of the evaluation in Section 5.

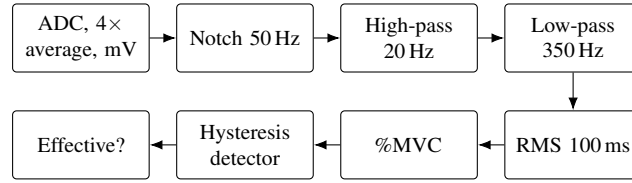


Figure 4: Signal chain from the pin to the repetition verdict.

RMS envelope. With $x[n]$ the filtered signal and $N = 100$ samples (100 ms),

$$E[n] = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} x[n-k]^2}. \quad (1)$$

It is implemented as a running sum in double precision, so it does not drift over hours at 1 kHz.

3.3 Calibration and normalisation

Pressing A starts a calibration. The user relaxes for 3 s, which gives the resting envelope E_{rest} as the mean over the window, then contracts as hard as possible for 3 s, which gives E_{MVC} as the maximum. Calibration is accepted only if $E_{\text{MVC}} > 2E_{\text{rest}} + 1$ mV; otherwise the screen reports a failure, which in practice means an electrode has come off or the user did not squeeze. Activation is then

$$a[n] = \max\left(0, \frac{E[n] - E_{\text{rest}}}{E_{\text{MVC}} - E_{\text{rest}}}\right). \quad (2)$$

3.4 Repetition detector

A repetition starts when a rises above 0.30 and ends when it falls below 0.15; the gap between the two stops the detector chattering around a single level. Bursts shorter than 200 ms are discarded as artefacts, a 300 ms refractory period prevents double counting, and an 8 s limit ends a runaway

```

state IDLE:
    if a >= on and (no previous rep or t - t_end >= 300 ms):
        state = ACTIVE; t0 = t; peak = a
state ACTIVE:
    peak = max(peak, a)
    if a < off or t - t0 > 8 s:
        state = IDLE; t_end = t
    if t - t0 >= 200 ms:
        report rep(start = t0, duration, peak, mean)
        effective = (peak >= target)

```

Figure 5: Repetition detector. Thresholds on and off are 0.30 and 0.15 of MVC for curls (0.28 and 0.14 for squats).

hold. Each repetition is recorded with its peak, mean and duration, and is *effective* if its peak reaches the exercise target: 0.60 of MVC for curls and 0.54 for squats. These targets are parameters set in `config.h`, not physiological claims. At the end of a set the device shows the repetition count, the effective count, the mean peak and the least-squares slope of peak against repetition number; a falling slope suggests fatigue.

3.5 Firmware structure

The sampler task posts completed repetitions to a queue. The user interface runs on core 1: button A calibrates, B starts and stops a set, C changes exercise. A live activation bar carries a marker at the target; each completed repetition flashes green or red and the motor vibrates for 150 ms if it was effective. Drawing can take tens of milliseconds without costing a sample. The optional cloud link publishes the activation at 2 Hz and the two repetition counters when they change; credentials are read from a `secrets.h` file that is not committed. All of the signal processing is in one header, `emg_dsp.h`, with no dynamic allocation and no Arduino dependency.

4 Verification Protocol

Parity test. `tests/test_pipeline.cpp` includes `emg_dsp.h` unchanged and runs it over a CSV of samples. A Python implementation of the same filter equations, window and detector, written separately, runs on the same file. We compare the envelopes sample by sample and the detected repetitions with their flags.

Simulated sessions. Each session lasts under a minute: 3 s of rest, two maximal contractions, then ten repetitions of which two are deliberately weak and must not count as effective, so there are eight effective repetitions per session. The EMG is band-limited noise (20–450 Hz) whose amplitude follows a smooth activation envelope with a slow decay over the set. We add 50 Hz hum at 20–100% of the signal level, a 1.65 V DC offset, baseline wander, ADC noise and three 80 ms motion spikes of up to 700 mV in the exercise window. Subject gain is drawn log-normally with a spread of about 35%.

Comparison detector. The simplest alternative is a fixed threshold: a repetition is counted when one raw sample exceeds a fixed level, followed by a 600 ms lock-out. We set the level, 2.37 V at the ADC pin, by hand for a nominal subject of gain 1.

Detector	TP	FP	FN	Precision	Recall	F1
FitPal chain (calibrated, %MVC)	480	5	0	0.990	1.000	0.995
Fixed threshold, 600 ms lock-out	227	100	253	0.694	0.473	0.563

Table 1: Effective-repetition detection over 60 simulated sessions with eight effective repetitions each.

Scoring. A detection is a true positive if it falls within -300 to $+1200$ ms of the start of a not-yet-matched effective repetition. We report precision, recall and F1 over all 60 sessions (480 effective repetitions).

5 Results

5.1 Parity

On the test session the envelopes of the C++ header and the Python reference differ by at most 0.0076 mV, which is single-precision rounding. Both detectors return the same ten repetitions with the same effective flags.

5.2 An example session

Figure 6 shows one simulated session. The three motion spikes are visible in the raw trace and in the envelope; none is reported as a repetition, because each is shorter than the 200 ms gate. Both weak repetitions are detected and marked as below target.

5.3 Detection over 60 sessions

Table 1 gives the totals. The right panel of Figure 7 explains the gap. The fixed threshold finds none of the effective repetitions at the lowest gains, because the signal never reaches its level, and more of them as gain rises, at which point it starts to count spikes and weak repetitions. The calibrated chain finds all eight at every gain.

6 Discussion

Why a fixed threshold fails. It makes the decision depend on three things the user does not control: electrode placement, skin condition and muscle mass. Hum and spikes then act on top of that. A threshold tuned on one person is correct for that person only. Normalising to a measured maximum moves the unknown out of the decision and into a six-second step the user can repeat when the electrodes are replaced.

Practical consequences. Calibrate again whenever electrodes are repositioned. Set the mains frequency to the local grid. If the Grove module turns out to output an envelope rather than raw EMG, the notch and high-pass stages should be disabled in `config.h`; the rest of the chain is unchanged.

What this study does not establish. The signal model is ours, so high scores are partly a property of the model. The fixed-threshold detector shows how a threshold behaves when gain varies; it is not a measurement of any existing product. No recordings from the physical sensor have been scored. The sketch compiles for the M5Stack Core2 with ESP32 Arduino core 2.0.17 and the M5Core2 library

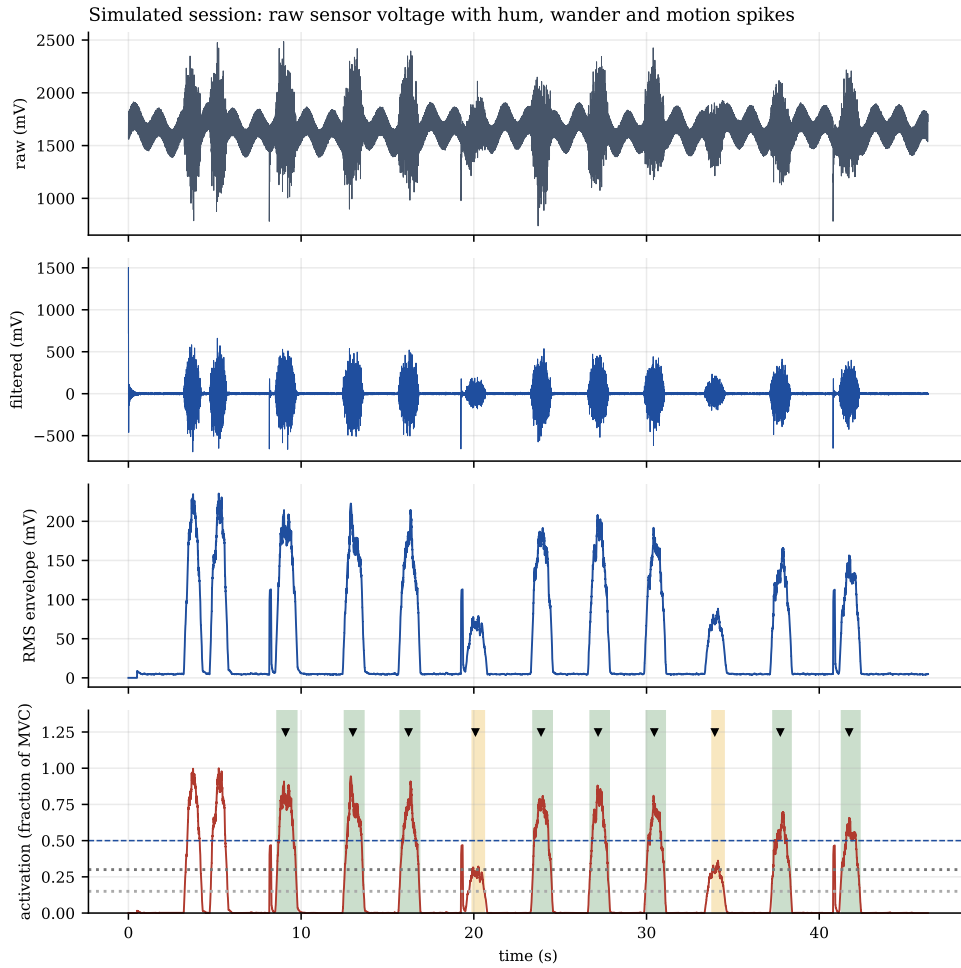


Figure 6: One simulated session. Green bands are effective repetitions and amber bands are detected repetitions below target. Triangles mark ground truth. Dotted lines are the 0.30 and 0.15 thresholds and the dashed line is the 0.50 target used in the simulation.

(about 460 kB of flash, 25 kB of static RAM), without the cloud option; the cloud variant was not compiled, and the firmware has not been run on the board. The effectiveness targets of 0.60 and 0.54 of MVC are working values and carry no physiological claim. The fatigue indicator is a slope of peaks and has not been compared with any other measure.

Next steps. Record raw 1 kHz millivolt streams to SD with video as the reference for repetition count, from several users, both exercises and at least two electrode placements; score detection against the video; measure the stability of E_{MVC} across days; and run the firmware on the board, including the cloud option.

Conclusion. The device turns a noisy surface-EMG voltage into a per-repetition verdict that is expressed relative to each user’s own maximum. The signal chain that runs on the board is the one that was tested, it matches an independent implementation to 0.0076 mV, and on simulated sessions that include hum, offset, wander, spikes and gain spread it is far more reliable than a fixed threshold. Whether that carries over to people is a question for recorded data.

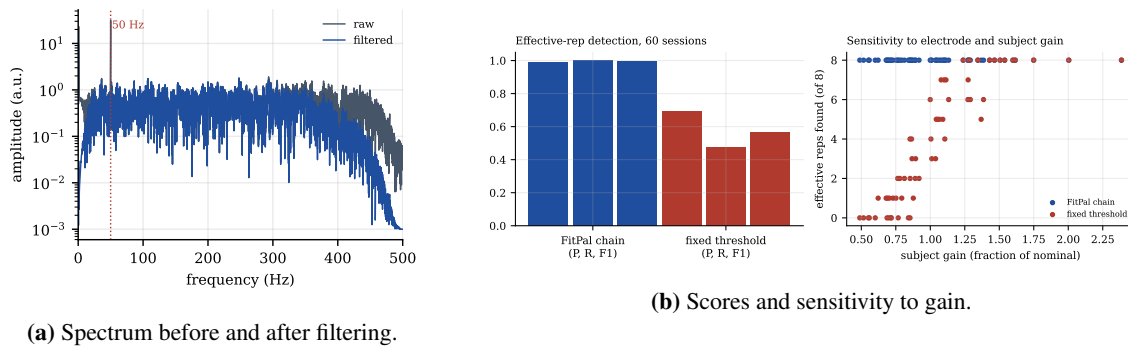


Figure 7: Filter effect and detection performance over 60 simulated sessions.

Code and data availability

The firmware, the PC test harness, the simulation and the scripts that generate every figure and number in this report are in the repository accompanying it (Appendix B).

References

- Robert Bristow-Johnson. Cookbook formulae for audio equalizer biquad filter coefficients. <https://webaudio.github.io/Audio-EQ-Cookbook/Audio-EQ-Cookbook.txt>. Accessed 2026.
- Adrian Burden. How should we normalize electromyograms obtained from healthy participants? What we have learned from over 25 years of research. *Journal of Electromyography and Kinesiology*, 20(6):1023–1035, 2010.
- Mario Cifrek, Vladimir Medved, Stanko Tonković, and Sasa Ostojić. Surface EMG based muscle fatigue evaluation in biomechanics. *Clinical Biomechanics*, 24(4):327–340, 2009.
- Edward A. Clancy, Evelyn L. Morin, and Roberto Merletti. Sampling, noise-reduction and amplitude estimation issues in surface electromyography. *Journal of Electromyography and Kinesiology*, 12(1):1–16, 2002.
- Carlo J. De Luca. The use of surface electromyography in biomechanics. *Journal of Applied Biomechanics*, 13(2):135–163, 1997.
- Hermie J. Hermens, Bart Freriks, Catherine Disselhorst-Klug, and Günter Rau. Development of recommendations for SEMG sensors and sensor placement procedures. *Journal of Electromyography and Kinesiology*, 10(5):361–374, 2000.
- Roberto Merletti and Philip A. Parker. *Electromyography: Physiology, Engineering, and Non-Invasive Applications*. Wiley-IEEE Press, Hoboken, NJ, 2004.

A Parameters

Stage	Parameter	Value
Sampling	rate / reads averaged per sample	1 kHz / 4
Notch	frequency / Q	50 Hz (60 Hz selectable) / 30
High-pass, low-pass	corner frequencies	20 Hz, 350 Hz
Warm-up	discarded samples	500
Envelope	window	100 ms
Calibration	rest / maximal contraction	3 s / 3 s
Detector	on / off threshold (curl)	0.30 / 0.15 MVC
Detector	minimum duration / refractory / maximum	200 ms / 300 ms / 8 s
Effectiveness	target (curl / squat)	0.60 / 0.54 MVC
Cloud	activation publish rate	2 Hz

Table 2: Parameters of the signal chain as set in `config.h` and `emg_dsp.h`.

B Reproducibility

From the repository root, with `numpy`, `scipy`, `matplotlib` and `g++` installed:

```
python3 analysis/simulate_and_evaluate.py
```

The script builds the C++ test, runs both implementations, writes `analysis/output/summary.json` and regenerates the three simulation figures. Sessions are seeded, so the numbers in this report are reproduced exactly.